

VAR-Turbo: Unlocking the Potential of Visual Autoregressive Models through Dual Redundancy

Xujiang Xiang¹, Fengbin Tu^{1,2,*}

¹ The Hong Kong University of Science and Technology, Hong Kong, China

² AI Chip Center for Emerging Smart Systems, Hong Kong, China

xxiangaf@connect.ust.hk, *Corresponding Author (fengbintu@ust.hk)

Abstract—Image synthesis task has recently drawn enormous attention from both the academia and the industry due to the recent advancements of generative models, which now can generate photorealistic images with conditional words from human. Among the generative models for image synthesis task, Visual Autoregressive (VAR) model is a promising avenue due to its strong scalability. Nevertheless, its exorbitant computational cost poses a formidable obstacle to widespread adoption. To this end, we propose a dedicated software/hardware co-design framework dubbed VAR-Turbo for unlocking the potential of VAR models. Specifically, in the software level, we propose a Draft-Free Parallel Decoding scheme by exploiting the Image Redundancy, which can decrease the sample steps by $> 80\%$, and a combination of Token Aggregation and Dynamic Bypass that capitalizes on the Model Redundancy introduced by the generative Transformer to reduce the computational load by $> 60\%$. In the hardware level, we propose a dedicated accelerator featuring 1) A Unified Attention Core and 2) Radix Sort Core, which can support the aforementioned algorithm pipeline seamlessly and efficiently. Under the collaborative design and synergy of software and hardware, VAR-Turbo achieves averagely 5047.4x, 210.3x, 6.1x, 3.8x speedups and 24818.2x, 423.5x, 6.0x, 7.8x energy-efficiency improvements over Xeon 8168 CPU, Nvidia V100, ViTCoD and AdapTiv, while maintaining the generation quality.

I. INTRODUCTION

Deep image synthesis, the process of generating images using deep learning, has advanced significantly in recent years [8], [73], [90], with Diffusion model [34], [42], [56], [59] and Visual Autoregressive (VAR) model [11], [65], [74] emerging as two dominant paradigms (see Fig. 1a). Diffusion models utilize a pair of Markov chains: a forward chain that gradually injects noise into data, and a reverse chain that denoises the data to produce images [58] [64]. In contrast, VAR models generate high-fidelity images via next-token prediction, successively predicting each pixel conditioned on previously generated pixels in a manner analogous to large language models (LLMs) [19] [28]. While both achieve high-fidelity synthesis through iterative refinement, VAR models exhibit two superior capabilities over Diffusion models: (1) **Scale-out: Seamless multi-modal integration**. VAR models can be seamlessly integrated with LLMs to facilitate multi-modal applications [20], [29], [83] due to their shared autoregression mechanism and unified main architecture. Empirically, as indicated in Fig.1b left, VAR-based multi-modal models consistently achieve higher GenScore than Diffusion-based counterparts (e.g., OpenAI’s GPT-4o [35] versus SD- and FLUX-Family [64] [5]). (2) **Scale-up: Power-law scaling behavior**. VAR models exhibit

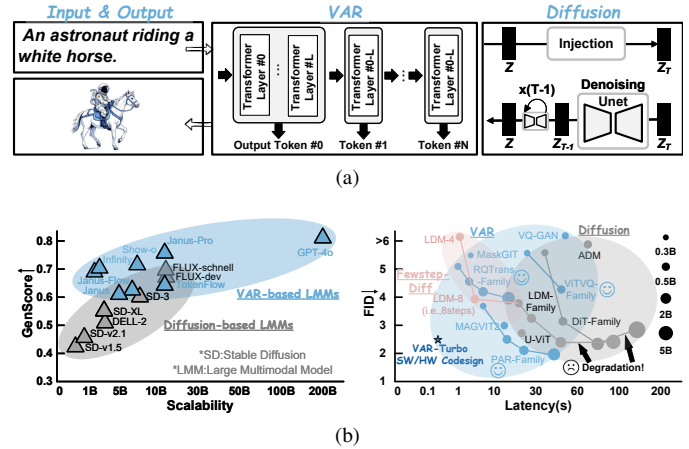


Fig. 1: (a) Two main generative models for image synthesis. (b) VAR models show superior scale-out (left) and scale-up (right) characteristics than Diffusion models. Here, GenScore (The higher, the better) is a metric measured on Geneval [27]. FID [33] (The lower, the better) is a metric measured on ImageNet. GenScore is quoted from [80]; FID and Latency is measured locally on our V100 GPU. Data Precision: FP32.

power-law scaling laws similar to those observed in LLMs [32], indicating that their capacities improve with the scaled-up dataset size, model size and compute budget. As shown in Fig. 1b right, two prevalent Diffusion models, DiT and LDM, do not show higher generation quality beyond 2B. In contrast, three popular VAR models all demonstrate the desired scaling-up feature. This is due to the fact that from the mathematical perspective (VAR: $Img = \prod_{t=1}^T p(x_t|x_1, x_2, \dots, x_{t-1})$, where x_t is the current pixel to be predicted [52]), VAR models inherently have a strong “visual context learning” capability, which is increasingly critical as the model is scaled up, as highlighted in OpenAI’s report [32].

Nevertheless, despite VAR models’ remarkable efficiency in both model scaling-out and scaling-up, their prohibitive computational cost remains a critical barrier to widespread adoption. This cost manifests at two levels: (1) **Inter-Iteration**: VAR models decode one visual token per iteration, requiring 256-4K steps to generate a single image; (2) **Intra-Iteration**: Each iteration invokes a generative Transformer whose computational complexity grows quadratically with the input sequence length. Consequently, prior VAR models typically take 10-60 seconds to generate a single image (see

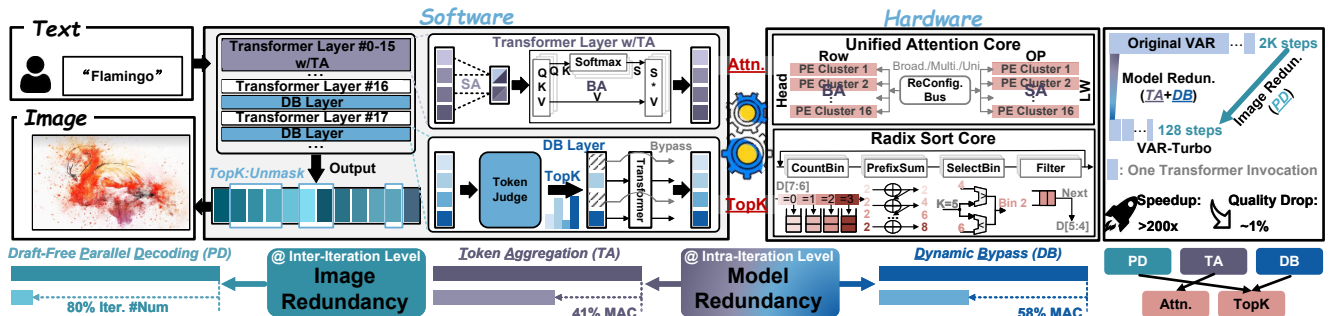


Fig. 2: An overview of VAR-Turbo, a software-hardware co-design framework dedicated to VAR models.

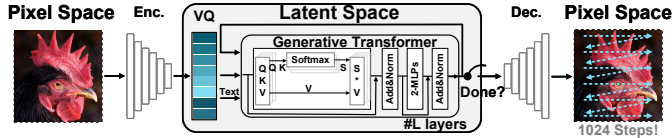


Fig. 3: An overview of the visual autoregressive model.

Fig. 1b right). At the inter-iteration level, speculative-decoding frameworks employ a draft model to decode multiple tokens per step [53] [24] [7]. Yet these methods yield only 2–4 tokens per step, far below the parallelism required for high-resolution images. At the intra-iteration level, a substantial body of work has sought to accelerate Transformers via a unified manner, e.g., unified sparse pattern, [14], [17], [46], [51], [61], [75], [77], [85]. However, these approaches fail to account for layer-wise variations in learning capabilities. This oversight results in the neglect of significant potential for further acceleration, as layer-specific characteristics that could be leveraged for efficiency gains remain unaddressed. To sum up, no prior work can fully unleash VAR models’ acceleration potential at both the inter- and intra-iteration levels.

This study identifies and exploits two substantial and orthogonal redundancies: **Image Redundancy at the Inter-Iteration Level** and **Model Redundancy at the Intra-Iteration Level**, aiming to optimize the overall latency of VAR models. First, images exhibit far richer spatial redundancy compared to low-redundancy languages [23] [88]. Second, the generative Transformer in VAR models demonstrates a heterogeneous distribution of redundancy levels: 1) In shallow layers, the model is less redundant and acquires diverse information. 2) In the deep layers, the model becomes highly redundant and shows inertia in learning new information. These two distinct regimes are designated as the “Learning Region” and the “Inert Region”, respectively (see Sec. II-C).

To leverage the opportunities of dual redundancy while synergizing corresponding algorithm- and accelerator-level innovations, we propose VAR-Turbo, the first software–hardware co-designed framework that accelerates VAR models at both the inter- and intra-iteration levels. VAR-Turbo delivers over 200× speedup for VAR models while incurring negligible quality degradation (< 1%; see Fig. 2 right). Specifically, our key contributions are as follows:

- To the best of our knowledge, this work constitutes the first systematic attempt to *both theoretically and empiri-*

cally characterize Image and Model Redundancy in VAR models, and to further reveal two distinct learning regimes that emerge as the Transformer layers are stacked. Guided by these findings, **in the software level**, we introduce three optimizations: 1) Draft-Free Parallel Decoding, 2) Token Aggregation and 3) Dynamic Bypass. First, at the inter-iteration level, Draft-Free Parallel Decoding exploits spatial image redundancy to enable unmasking TopK “confident” tokens per iteration (up to 64 tokens) without relying on an auxiliary Draft Model, reducing the number of sampling steps by > 80%. Second, at the intra-iteration level, Token Aggregation and Dynamic Bypass leverage model redundancy to lower the computational burden of the Transformer. Specifically, in the Learning Region, Token Aggregation employs a hierarchical attention mechanism (Small Attention followed by Big Attention) to cut attention MAC-ops by 41%. In the Inert Region, Dynamic Bypass leverages model’s collapsed learning ability by skipping attention and MLP layers for TopK uninformative tokens (Fig. 2, middle), yielding an additional 58% MAC reduction.

- **In the hardware level**, as shown in Fig. 2 right, we pinpoint two primary acceleration bottlenecks: 1) the heterogeneity between Big & Small Attention present by Token Aggregation, and 2) the large-scale sorting operations required by Parallel Decoding and Dynamic Bypass (a quantitative characterization of these challenges is provided in Sec. IV-A). Targeting the two bottlenecks, we propose a dedicated accelerator with a Unified Attention Core and a Radix Sort Core to fully unleash the potential of VAR-Turbo’s software optimizations.
- Extensive experiments and ablation studies consistently validate the advantages of our proposed VAR-Turbo framework, yielding averagely 5047.4x, 210.3x, 6.1x, 3.8x speedups and 24818.2x, 423.5x, 6.0x, 7.8x energy-efficiency improvements over Xeon Platinum 8168 CPU, Nvidia V100, ViTCoD and AdapTiV, while maintaining the generation quality.

II. BACKGROUND AND MOTIVATION

Section I established that the prohibitive computational cost of Visual Autoregressive (VAR) models is a critical barrier to their adoption. We posited that this challenge can be addressed by exploiting two orthogonal redundancies inherent

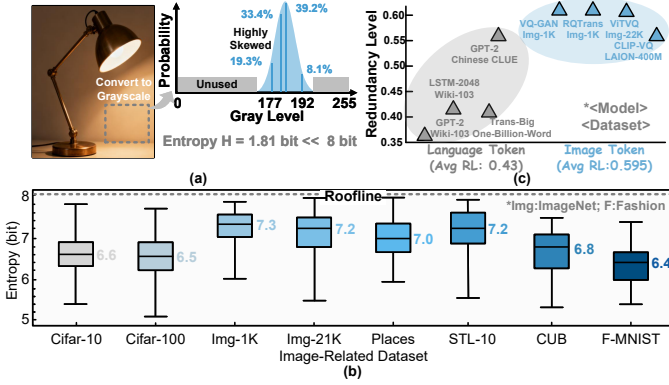


Fig. 4: (a) The reason why entropy can represent image spatial redundancy. (b) The information entropy of 32k images/dataset from the training set. Data Precision is INT8. To facilitate entropy calculation, all images are transformed into tensors for batched parallel computing on GPU (batch size = 512). (c) The statistics data about the Redundancy Level for language token and image token (also 32k samples).

in the generation process. This section lays the necessary groundwork for our approach. We first provide a detailed background on the standard VAR model pipeline to precisely identify its core computational bottlenecks. Following this, we will theoretically and empirically characterize the **Image Redundancy** and **Model Redundancy** that our VAR-Turbo framework is designed to leverage.

A. Visual Autoregressive Model

VAR models generate images by sequentially predicting each pixel based on preceding pixels, mirroring the “next-token prediction” approach in LLMs. The typical VAR models usually adopt a two-stage approach: 1) **Tokenization**, and 2) **Generation** (see Fig. 3). Specifically, in the first stage, an encoder (**Enc.**) tries to compress images from the pixel space to the latent space. Then, Vector Quantization (**VQ**) in the first stage will maintain a learned codebook, which is used to discretize continuous latent tokens [21]. The discretization process is similar to the nearest neighbor search, which will return the index of the query token’s closest code. Then, the discrete visual tokens will be concatenated with the text tokens which are used to condition the image generation process. Afterwards, in the stage 2, the generative Transformer generates one token per iteration in a raster-scan way. Upon iteration finished, the decoder (**Dec.**) in the first stage will turn the generated implicit visual tokens back to the explicit meaningful pixels. Hence, the inefficiency of VAR models stems from two bottlenecks: 1) **One-token-per-iteration decoding**, and 2) **A compute-heavy Transformer at every iteration**.

B. Image Redundancy

In this section, we will analyze and quantize the image redundancy from the angle of Information Theory [67], i.e., Entropy. Taking a gray scale image as an example, the entropy is calculated as $H = -\sum_{i=1}^{256} p_i \log p_i$. Besides, the maximum

entropy $H_{\max}$ is achieved when gray levels are evenly distributed, i.e., $P_i = \frac{1}{256}$, yielding $H_{\max} = \log(256) = 8 \text{ bit}$ [67]. As shown in Fig. 4 (a), we select a patch featuring highly similar to measure its entropy, which is only 1.81 bit, far less than its max entropy, 8 bit. In other words, the patch could be represented with only 1.81 bit/pixel instead of the raw 8 bit/pixel. The 6.19 bit gap is known as *information redundancy* [67]. This redundancy arises because when pixel values in a patch are highly similar, they occupy only a narrow range of the full spectrum (see the blue curve in Fig. 4 (a), only spanning from 177 to 192), resulting in a highly skewed distribution with broad regions unused. Therefore, based on the above analysis, entropy can serve as a theoretically grounded proxy to quantify the image redundancy. Besides, we can also conclude that *the more redundant the image is, the more skewed its pixel distribution, and thus the lower its entropy*. Next, we’ll prove that image is more redundant than language both before and after tokenization.

Since the max entropy $H_{\max}$ varies case by case, we use Redundancy Level (RL), defined as $1 - \frac{H}{H_{\max}}$, to fairly quantify redundancy (the higher the RL, the greater the redundancy). At the input-modality level, Fig. 4 (b) shows that most images yield entropy $H \in (6.4, 7.3)$, giving image’s RL $\in (0.09, 0.20)$. In contrast, according to Shannon’s letter entropy [67], English’s RL is $1 - \frac{9.96}{10} = 0.004$ and Chinese’s RL is $1 - \frac{10.77}{11.46} = 0.06$. Thus, image is more redundant than language at the input-modality level. Furthermore, we dive into the token level’s redundancy for image and language (Fig. 4 (c)). At this level, $H_{\max}$ is computed as $\log_2(\text{Codebook_Size})$. Empirically, across a wide range of models and datasets, the average RL for image tokens is 0.595, whereas for language tokens, it is only 0.43. Consequently, we conclude that both at the input-modality level and the token level, image exhibits greater redundancy than language.

In terms of the optimization leveraging redundancy to reduce latency, Speculative Decoding in NLP employs distilled small language models as Draft Models to draft multiple candidate tokens in parallel, then verifies the candidate tokens by LLM and only accepts the Top-K confident tokens as the final LLM’s output [53]. This method has two inherent limitations: the extra cost of running Draft Models and low parallelism for parallel decoding (2-3 tokens per iteration). Motivated by the observation that images intrinsically contain richer redundancy than language, we hypothesize that VAR models can decode more correlated visual tokens per iteration without relying on draft models. To this end, we introduce **Draft-free Parallel Decoding**, a method designed to fully unleash the parallel decoding potential of VAR models, thereby achieving substantial acceleration at the inter-iteration level (Sec. III-A).

C. Model Redundancy

Model redundancy refers to the redundancy stemmed from the Transformer, more specifically, from the “low-pass” attribute of the attention. Despite its prevalence in Transformers, none of prior works formally identify this kind of redundancy and mathematically prove it. This section first proves the “low-

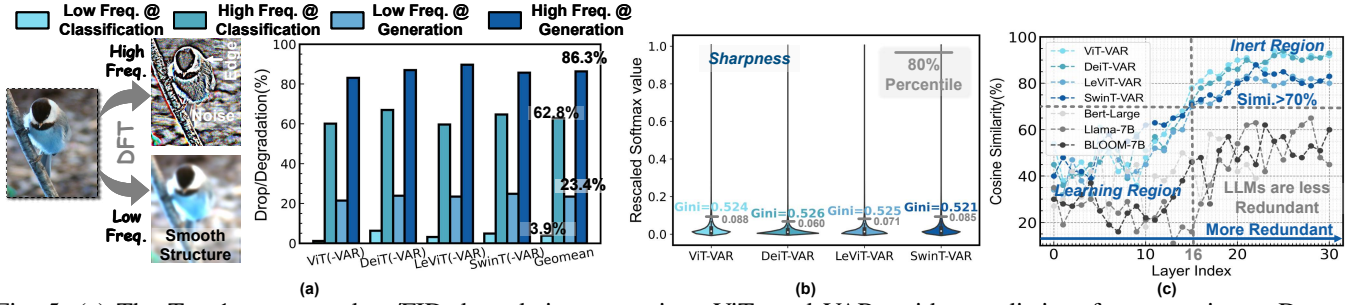


Fig. 5: (a) The Top-1 accuracy drop/FID degradation on various ViTs and VARs with two distinct frequency input. Dataset: ImageNet-1K. (b) The violin plot about the statistics for the rescaled value in attention maps on various VAR models (**Intra-Layer Similarity**). (c) The cosine similarity of the attention maps from adjacent layers (**Inter-Layer Similarity**).

pass” nature of attentions and elaborates on its role as the underlying cause of model redundancy. Second, it reveals the layer-wise variations in such redundancy.

The attention is formulated as $O = \text{Softmax}(Q \times K^T) \times A$. Due to the row-wise dependency of Softmax [39], we use the Gustavson’s algorithm to conduct matrix multiplication [91]. So, for every row of O , we have $o = \text{Softmax}(q \times K^T) \times A = \sum_{i=1}^N (e^{q \cdot k_i^T} v_i) / \sum_{i=1}^N e^{q \cdot k_i^T}$. Then, we assume that vectors in Q and K are normalized to have a fixed L2 norm, denoted as C_1 and C_2 respectively, we have:

$$\begin{aligned} q \cdot k_i^T &= -\frac{1}{2}(\|q - k_i\|_2^2 - C_1^2 - C_2^2) \\ \Rightarrow o &= \sum_{i=1}^N (e^{-\frac{\|q - k_i\|_2^2}{2}} v_i) / \sum_{i=1}^N e^{-\frac{\|q - k_i\|_2^2}{2}} \\ \Rightarrow o &= \frac{\int e^{-\frac{\|q - k\|_2^2}{2}} (\sum_{i=1}^N \delta(k - k_i) v_i) dk}{\int e^{-\frac{\|q - k\|_2^2}{2}} (\sum_{i=1}^N \delta(k - k_i)) dk} \\ \Rightarrow o(q) &= \text{Gau}(q, 1) * I(q, K, V) / \text{const}(q) \end{aligned} \quad (1)$$

It is noticeable that $\delta(x)$ is the Dirac delta function [30], $\text{Gau}(x)$ is the Gaussian kernel featuring low passing [63], $I(q, K, V)$ is a sparsely sampled multi-dimensional signal and $\text{const}(q)$ is a constant function with respect to q . Therefore, the attention mechanism can be conceptualized into “an input signal $I(q, K, V)$ is firstly filtered by a low-pass Gaussian kernel $\text{Gau}(q, 1)$, and then scaled by a scalar $\text{const}(q)$ ”. Specifically, after tokenization, the attention acts as a low-pass filter: tokens containing low-frequency image content are propagated smoothly, whereas those with high-frequency content are attenuated or even truncated. Consequently, the high-frequency part of image characterized by sharp transitions like edges and noise (Fig. 5 (a), leftmost) are progressively overlooked as attentions stack, while the low-frequency part like smooth structures are well preserved and processed. If this conjecture holds, feeding only the high-frequency component to a Transformer should degrade accuracy more severely than feeding only the low-frequency component. To test this, we use DFT (filter size = 180) to split images into high- and low-frequency parts and test them separately on ViT, LeViT, DeiT, and SwinT. As shown in Fig. 5 (a), the low-frequency part incurs a 3.9% accuracy drop @ classification, while the high-frequency part causes a 62.8% drop. For VAR models

with these Transformers as backbones, the FID degradation is 23.4% for the low-frequency part and 86.2% for the high-frequency part. These results confirm that attention intrinsically acts as a low-pass filter.

Consequently, the attention will yield a low-frequency-dominated, smoothed representation, which underpins the observed model redundancy. As depth increases, this low-pass characteristic accumulates—akin to cascading low-pass filters—causing the attention maps in deeper layers to converge toward near-identical patterns. Empirically, we uncover two manifestations of this redundancy: **(1) Intra-Layer Similarity**. As shown in Fig. 5 (b), attention maps exhibit a pronounced polarization: broad regions of smoothness punctuated by sparse and sharp spikes, with 80% value clustered below 0.06-0.088. Therefore, the Gini coefficient for all four VAR models exceeds 0.5. Such drastic skewness confirms a high degree of similarity in the attention. **(2) Inter-Layer Similarity**. We quantify the resemblance between consecutive-layer attention maps using cosine similarity [44]. Fig. 5 (c) shows that for VAR models, the inter-layer similarity ranges from 32% to 93%, whereas for LLMs it ranges only from 11% to 63%. Hence, the model redundancy is markedly more pronounced in VAR models than in LLMs. In addition, the distribution of inter-layer similarity splits into two distinct regimes: **(1) Learning Region (layer index < 16)**. In this region, the cosine similarity fluctuates widely, indicating that the model is acquiring diverse information. **(2) Inert Region, not shown in LLMs (layer ≥ 16)**. The cosine similarity in this region stabilizes at or above 70%, revealing consistent, redundant attention patterns. This implies that VAR models within this region exhibit very limited capacity to acquire new features. It is worth noting that the boundary between the learning region and the inert region in this study is defined by the observation that none of the baseline LLMs exhibit inter-layer cosine similarities exceeding 70%. In contrast, layers of VAR with indices equal or greater than 16 demonstrate inter-layer similarities above this threshold. Consequently, the 70% threshold is not arbitrary but is grounded in the distributional properties of the baseline LLMs’ inter-layer similarity.

Based on the above observations, two key insights emerge. First, the significant intra- and inter-layer redundancy in VAR models indicates substantial potential for reducing the com-

putational budget of their Transformers at the intra-iteration level. Second, leveraging the heterogeneous distribution of inter-layer redundancy identified earlier, a hybrid optimization strategy is adopted. In the Learning Region, where inter-layer similarity is limited, a Token Aggregation (TA) scheme is employed to capture intra-layer similarity: Small Attentions first merge tokens, followed by a Big Attention for long-distance modeling (Sec. III-B). In the distinct Inert Region, high inter-layer similarity enables more aggressive acceleration via Dynamic Bypass (DB), which skips entire Transformer layers for tokens classified as uninformative (Sec. III-C).

III. PROPOSED VAR-TURBO ALGORITHM

The analysis in Section II provided a theoretical and empirical foundation, characterizing Image Redundancy as an inter-iteration opportunity and Model Redundancy as an intra-iteration opportunity with distinct “Learning” and “Inert” regions. Building upon these insights, this section introduces the three core algorithmic optimizations of the VAR-Turbo software framework. We first present **Draft-Free Parallel Decoding (PD)**, which leverages image redundancy to reduce the number of sampling steps. We then introduce our hybrid strategy to exploit model redundancy: **Token Aggregation (TA)** to reduce computation in the Learning Region and **Dynamic Bypass (DB)** to aggressively skip operations in the Inert Region.

A. Draft-Free Parallel Decoding

Draft-Free Parallel Decoding (PD) comprises three core steps: 1) Initial Guess, 2) Confidence Calculation, and 3) Token Replacement (see Fig. 6 left and Algo. 1). **Initial Guess:** The generative Transformer will predict a visual token for each input token. Gumbel Sampling Trick is employed here to make this step differentiable (Line 6 in Algo. 1) [43]. **Confidence Calculation:** In this step, firstly, we will gather every predicted visual token’s probability, which stands for the confidence of the predicted token. Then, we will set the unmasked tokens’ confidence to $-\infty$ to make sure they will not be replaced by the predicted tokens in Step 3, since they’re already decoded. **Token Replacement:** In each iteration, we will set a pre-defined number of tokens to be unmasked to balance the generation speed and quality, which is controlled by the schedule function introduced in [54] (e.g., $r(t)$ in Algo. 1, the mask ratio in each step). Then, we use the Sort_TopK operator to determine the TopK confident tokens (i.e., tokens with the TopK highest probability), which are replaced by the predicted visual tokens (Line 15 in Algo. 1). In addition, those “unconfident” tokens or already decoded tokens and their corresponding Mask indicators remain unchanged (Line 17 in Algo. 1).

Fig. 6 right shows that while parallel decoding in LLM decodes 2–4 tokens sequentially [53], our Draft-Free Parallel Decoding enables unstructured decoding of significantly more tokens (up to 64) per iteration. This efficiency gain stems from leveraging the substantial spatial redundancy intrinsic to images, a characteristic not present in the linguistic data

processed by LLMs. Such a design also obviates the need for a separate draft model in VAR models, thereby further enhancing the overall efficiency [9], [18], [36], [79].

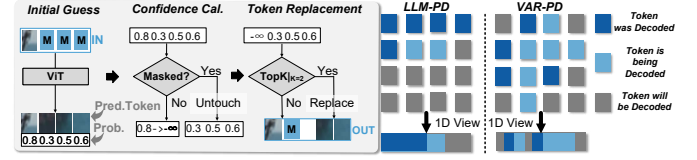


Fig. 6: An illustration of Draft-free Parallel Decoding (left) and a comparison between LLM-PD and VAR-PD (right).

Algorithm 1 Draft-free Parallel Decoding

Input: V: Visual Token, M: Mask Array, T: Iteration Step;
Output: IMG: Generated Image;

```

1:  $V_0 = \text{Concat}(\text{Text Token}, \text{Visual Token})$ 
2:  $M_0 = \text{ALL True}$ 
3: for  $t = 1$  to  $T$  do
4:   logits = ViT( $V_{t-1}$ )
5:   // Initial Guess
6:   prob = log(Softmax(logits)) + Gumbel Noise
7:   pred_ $V_t$  = argmax(prob)
8:   // Confidence Calculation
9:   Conf = prob.gather(idx=pred_ $V_t$ )
10:  Conf[ $\sim M_{t-1}.bool()$ ] =  $-\infty$ 
11:  // Token Replacement
12:   $K = N \times (1 - r(t))$ 
13:  Sort-TopK(Conf, K=K)
14:  if Conf  $\in$  TopK_Region then
15:     $V_t = \text{pred}_V$ ,  $M_t = \text{False}$ 
16:  else
17:     $V_t = V_{t-1}$ ,  $M_t = M_{t-1}$ 
18:  end if
19: end for
20: IMG = Decoder( $V_T$ )
21: Return IMG

```

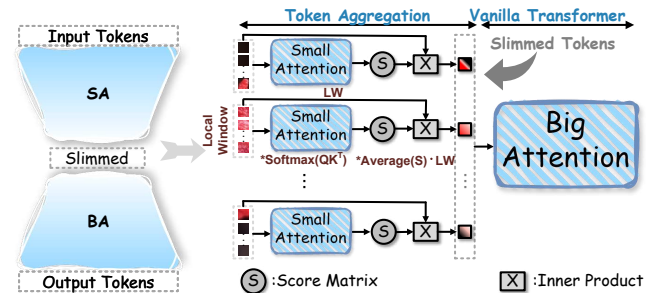


Fig. 7: A simplified illustration of Token Aggregation (TA).

B. Token Aggregation

Token Aggregation (TA) primarily consists of two attentions: Small Attention (SA) and Big Attention (BA). SA is a learnable lightweight attention to adaptively aggregate the neighboring tokens’ information into one representative token

and then BA is employed to conduct global modeling for those representative tokens generated by SA. Fig. 7 illustrates how TA works. First, the token sequence is segmented into multiple non-overlapped Local Windows (**LW**s), within which the visual tokens are sent to their respective SA modules. Here, an LW denotes the segmented local input tokens corresponding to an SA module (i.e., one LW $\leftrightarrow$ one SA). SA can be formulated as $Q_{lw}, K_{lw} = LW \times W_Q, LW \times W_K$; $S = \text{Softmax}(Q \times K^T)$. Each row of the score matrix $S = \text{Softmax}(Q \times K^T)$ is averaged to generate a coefficient vector whose length is LW.size. For each LW, the coefficient vector undergoes an inner-product with its corresponding input tokens to produce one final representative token. All representative tokens from SAs are then concatenated and sent to the regular BA module, such as multi-head self-attention. Notably, TA is applied only to shallow layers (layer #0-15), since these layers are in the Learning Region (see Fig. 5 (c)) and TA can greatly preserve the information. Finally, unlike prior hierarchical-attention methods such as BigBird [89], Longformer [4], and ViTCoD [85], TA provides two key advantages. First, TA avoids irregular computing, ensuring minimal hardware overhead. Second, it does not impose a fixed attention pattern on VAR models, thus preserving generation quality. For instance, BigBird and ViTCoD cause a 3.6% and 2.9% quality drop respectively on the DeiT-VAR model, while TA results in a quality degradation of less than 0.5%.

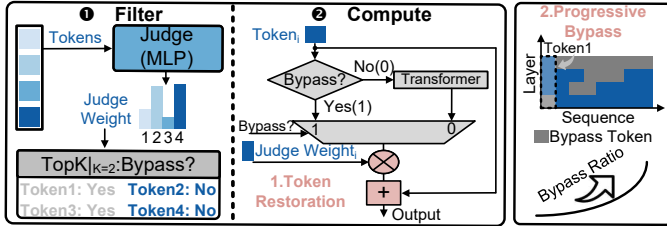


Fig. 8: An illustration of Dynamic Bypass (DB) and its two key features: 1) Token Restoration and 2) Progressive Bypass.

C. Dynamic Bypass

As depicted in Fig. 8, Dynamic Bypass (DB) consists of two stages. Stage ① **Filter**: Each input token is first sent into a lightweight MLP that outputs an importance score (or weight). A Top-K operation then retains only the K highest-scoring tokens; the remainder are marked for bypass. Stage ② **Compute**: Only the tokens with the highest TopK scores are forwarded to the Transformer layer. Therefore, DB optimizes for both Attention and FFNs in terms of the computational budget. In addition, DB has two unique features to safeguard model capability and further lower the computational load, respectively: **1) Token Restoration** (apricot-colored blocks in Fig. 8 middle). Tokens bypassed at the current layer are reinstated via element-wise multiplication and addition before sent to the next layer (i.e., $Token_i \times Judge Weight_i + Token_i$), mitigating cumulative information loss that could arise if previously bypassed tokens later become informative. For example, as shown in Fig. 8 right, Token1 which is bypassed in layer1 is considered informative in the subsequent three

layers. If we just discard Token1 from layer1 like prior early exit work, SpecEE [12], it will degrade generation quality by 1.6% @ DeiT-VAR. In contrast, DB only degrades quality by less than 0.5% due to the token restoration (see Sec. V-F). **2) Progressive Bypass**. Motivated by the monotonic increase in redundancy within the highly redundant Inert Region of VAR models (Fig. 5 (c)), DB gradually raises the bypass ratio as layers deepen (Fig. 8 right), saving more computations.

IV. PROPOSED VAR-TURBO ACCELERATOR

The software optimizations detailed in Section III—Draft-Free Parallel Decoding, Token Aggregation, and Dynamic Bypass—successfully reduce the theoretical iteration count and computational complexity of VAR models. However, these algorithmic innovations introduce unique and demanding execution patterns that are not efficiently addressed by existing hardware. Specifically, TA creates heterogeneous attention computations (Small and Big Attention), while PD and DB rely heavily on large-scale, variable-length sorting operations. This section presents the VAR-Turbo accelerator, a dedicated hardware architecture co-designed to execute this algorithm pipeline seamlessly and unlock its full acceleration potential.

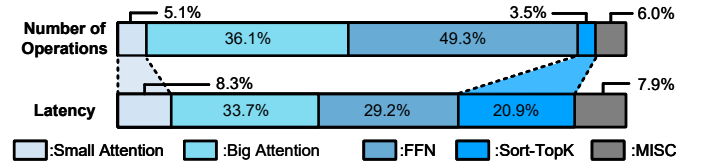


Fig. 9: #Operations and Latency breakdown results.

A. Hardware Challenges

First, Token Aggregation (TA) instantiates two heterogeneous attentions, i.e., Small Attention (SA) and Big Attention (BA). The SA features being lightweight and multi-threaded, while the BA is bulky and monolithic. In addition, SA, despite accounting for only 5.1% #Operations, consumes a relatively higher proportion of latency, reaching 8.3% (see Fig. 9), necessitating a unit to accelerate it. Presenting two separated attention cores for the two attentions respectively will decrease the hardware utilization. **Second**, the value K of TopK would be very large in variable-length input sequences for Draft-Free Parallel Decoding (PD) and Dynamic Bypass (DB), e.g., $K = 1936$ in $N = 4096$. As shown in Fig. 9, the Top-K operator, although contributing to only 3.5% #Operations, occupies a significant 20.9% latency. Prior TopK units predominantly adopt Bitonic Sort + Merge sort to arrange the input sequence [49], [61], [75], [77], which is highly efficient when K is small (e.g., $K_{max} = 64$). However, this efficiency drastically diminishes for large K values [66], as it involves numerous re-reads, re-sorts and re-writes for the input sequence, like Top64, Top128, up to Top1936. To efficiently address the above two challenges, respectively, VAR-Turbo accelerator introduces two architectural innovations, i.e., **Technique T1: Unified Attention Core** and **Technique T2: Radix Sort Core**.

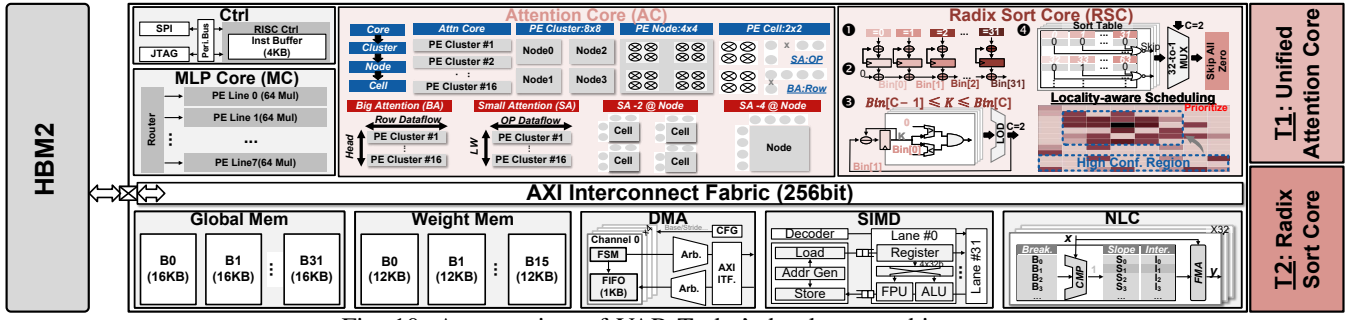


Fig. 10: An overview of VAR-Turbo's hardware architecture.

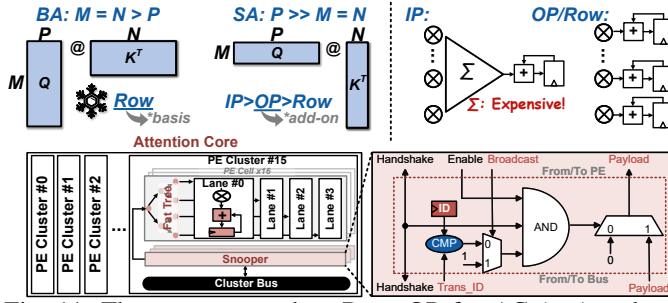


Fig. 11: The reason we adopt Row+OP for AC (top) and an overview of the unified Attention Core (bottom).

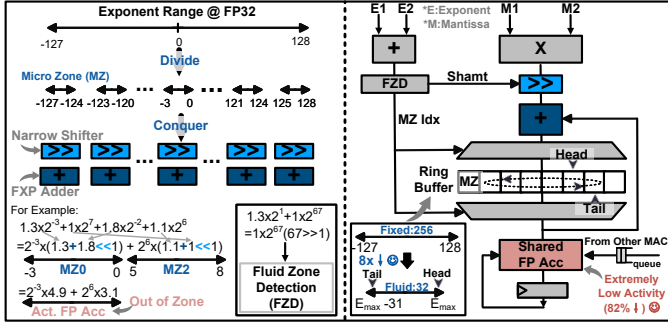


Fig. 12: The Divide-and-Conquer scheme (left) and the corresponding optimized MAC unit for Row+OP dataflow (right).

B. Architecture Overview

Fig. 10 outlines the architecture of VAR-Turbo accelerator. First, we architect the chip with drawing inspiration from NVDLA [22] and TPU [38]. We begin by adopting a bus-based topology similar to NVDLA. Then, following TPU's lead, our memory hierarchy includes a global memory for input and output, as well as a weight memory dedicated for the weight. Lastly, we construct a computation pipeline that includes a MLP Core (MC), an Attention Core (AC), a Radix Sort Core (RSC), a Non-Linear Core (NLC), and a SIMD Core to facilitate layer fusion [57] [39]. MLP Core comprises 8 PE Lines, with each PE Line consisting of 64 multipliers. This unit only handles the "activation-weight" operators [39]. SIMD Core is a vector co-processor, handling the element-wise operators. Non-Linear Core is inspired by NN-LUT [86], which leverages the piece-wise linear approximation to implement the non-linear operators. For ensuring a more

compact SOC schedule, we add a 24bit specialized field within instructions to explicitly specify the "Producer-Consumer" data dependency among different units, as detailed in [87].

C. Attention Core

Numerous studies have demonstrated the superiority of Row-wise (Row) dataflow for Transformers [39] [76] [47]. Thus, for the major kernel, BA, we adopt the Row dataflow, which is regarded as a basis dataflow. However, as shown in Fig. 11 top, the matrices for SA have a unique feature, i.e., $P \gg M/N$, where P =embedding.size and M/N =LW.size (LW.size $\in (2, 4)$, see Sec. V-F), which makes the inner product (IP) dataflow the most suitable. In addition, as shown in the top right of Fig. 11, IP favors the spatial adder tree and OP (outer product) and Row favor the temporal accumulator. Thus, if we adopt "IP + Row" to construct a unified architecture for AC, the hardware overhead will be skyrocketed due to the duality of adders (25.8% area $\uparrow$ and 30.4% power $\uparrow$ @1GHz). To make a reasonable trade-off, we opt for this combination, "OP for SA + Row for BA", which renders AC to improve the throughput by 29.2% with the overhead of only extra 8.5% area and 7.2% power, compared to NVDLA (a fixed-dataflow baseline). The reconfigurability of Attention Core can be seen in Fig. 10 top middle. In BA mode, each PE Cell manages the Row dataflow and each PE Cluster handles an attention head. In contrast, in SA mode, each PE Cell executes the OP dataflow. Additionally, when LW.size is 2, each PE Cell handles a local window; when LW.size is 4, each PE Node (i.e. four PE cells) processes a local window.

The key of the dataflow switching is to construct a flexible distribution bus, as shown in Fig. 11 bottom, which comprises two level of routing networks, i.e., **Snooper** and **Fat Tree**. **Snooper**. The packet in the Cluster Bus has three primary fields: ID, Type and payload. The ID field is 4b, with high 2b indicating the index of PE Node and low 2b indicating the index of PE Cell. The Type field indicates the data type, i.e., streaming data or stationary data. The payload field is the data to be consumed. During the packet transfer, Snoopers will snoop the packets on the Cluster Bus and then only transfers the packet with the same ID to the PE Cell. Thus, to support flexible transfer patterns, we need to configure the ID reg for each PE Cell in advance. **Fat Tree**. Fat Tree is a typical tree based network [45], which can distribute the payload from the Snooper to the computing lanes in a non-blocking manner.

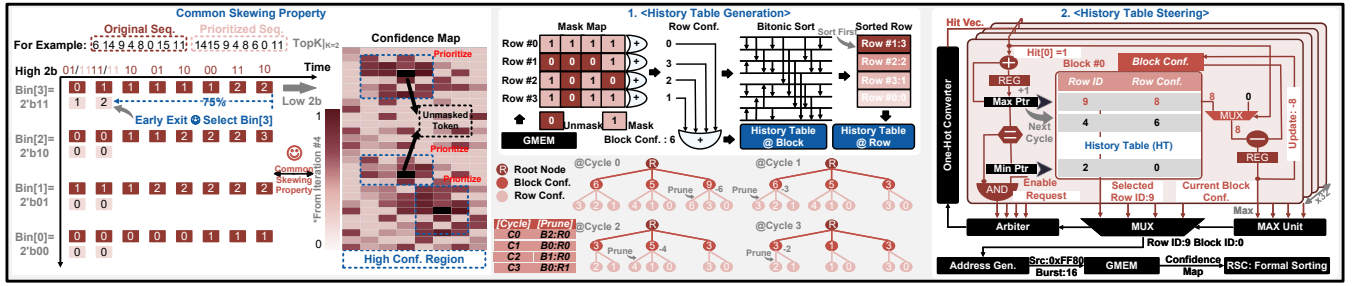


Fig. 13: The common skewing property between the Radix Select (RS) algorithm and the confidence map in PD (left). An overview of the Locality-aware Scheduling (LAS) scheme (middle and right).

However, Samsung pointed out that the accumulator-based PE consumes higher power than the adder-tree-based PE due to the high circuit activity of the FP accumulator [37] (34% $\uparrow$ @ TSMC+N28). To address this critical issue, we propose a **Divide-and-Conquer** scheme to minimize the activity of the energy-hungry FP accumulator, which is inspired by the Zone-based IPU in [40] [50]. First, as shown in Fig. 12 left, we divide the wide exponent range into multiple Micro Zones (MZs) and then employ the low-cost FXP adder and narrow shifter to conquer each MZ. Only in the event of an Out-of-Zone case, e.g., $2^{-3} \times 4.9 + 2^6 \times 3.1$ (see Fig. 12), is the area- and power-hungry FP Acc activated. Consequently, the activity of FP Acc is lowered by 82%. Further, we notice that the exponent range can be fluid (i.e., Fluid Zone Detection, FZD, see Fig. 12) due to the limited mantissa bit-width (e.g., 24bit for FP32), which can greatly narrow down the Exp detection range ($8 \times \downarrow$). For example, $1.3 \times 2^1 + 1 \times 2^{67} = 1 \times 2^{67}$, because the former number is completely shifted out in the mantissa alignment caused by the excessively large exponent difference, i.e., 66. Thus, as depicted in Fig. 12 right, we employ a Ring buffer to facilitate FZD. The head pointer points to the slot referring to the $[E_{max} - 3, E_{max}]$ MZ. When there is an incoming larger Exp than current E_{max} , we will store the larger Exp in the tail slot which now becomes the head slot. Working in this circular way, E_{max} can be fluid, i.e., detected in an on-the-fly way rather than fixed to 128. Additionally, thanks to the extremely low activity of FP Acc, we can share one area-hungry FP Acc across two MAC units. To address the collision issue where two multipliers attempt to occupy the same Shared FP Acc, each MAC unit is equipped with a queue. Given the low collision probability ($< 7\%$), the queue depth is only set to 2. Collectively, this Divide-and-Conquer scheme with FZD and a shared FP Acc reduces the power by 59% and area by 11% per pair of MAC units.

D. Radix Sort Core

Radix Sort Core (RSC) is based on the Radix Select (RS) algorithm [66] [69]. The basic idea of Radix Select is to split the M-bit elements into smaller d-bit digits, resulting in a small enough radix $r = 2^d$. Then, the amount of sorting iteration is a deterministic value, $\lceil \frac{M}{d} \rceil$, which is irrespective of K and the sequence length. After $\lceil \frac{M}{d} \rceil$ iterations, RS picks the K-th largest element and then we just apply another comparison pass to input sequence to accomplish the TopK operation. Therefore,

RS exhibits much higher efficiency for large K and long variable-length input sequences than the prior art like Merge Sort, which renders RS a perfect candidate for implementing the TopK operator in VAR-Turbo. The procedure of RS can be divided into four main steps: ① CountBin, ② PrefixSum, ③ SelectBin and ④ Filter (see Fig. 10). ① CountBin. Every bin will count the number of input elements with the same binary number (i.e., hit counts), which involves a set of parallel counters [71]. Then, ② PrefixSum. A set of parallel prefix adders [16] will accumulate the hit counts from all the previous bins up to the current bin. Afterwards, ③ SelectBin. SelectBin will identify the bin potentially containing the pivot (i.e., K-th element), which involves a set of parallel comparators. Then, the output of comparators will be sent to a Leading-One-Detector (LOD) to finally detect the Candidate Bin. At last, ④ Filter. Filter will exclude all elements whose binary numbers don't match the index of the Candidate Bin. Next, we will introduce a **Locality-aware Scheduling (LAS)** scheme to maximize the efficiency of the original RS algorithm.

First, as shown in Fig. 13 left, RS algorithm inherently exhibits the skewing property [69]. The sooner and the more frequently the input sequence populates the larger bins (e.g., Bin[3]), the faster the sorting completes. For the Prioritized Seq., number 14 and 15 have already hit Bin[3] twice in total, triggering an early-exit opportunity that cuts latency by 75% relative to the Original Sequence. Then, as stated in Sec. II-B, there is one unique feature of the TopK in VAR-Turbo: Tokens to be decoded are correlated spatially, which is visualized in the Confidence Map in Fig. 13 left. Masked tokens adjacent to unmasked ones tend to receive higher confidence scores. In short, the Confidence Maps are intrinsically skewed. To capitalize on this unique opportunity (i.e., **common skewing property**), we propose the LAS scheme to prioritize the sorting for these High Conf. Regions to minimize the latency.

As shown in Fig. 13 middle, LAS firstly generates History Tables (HTs) that are derived from the Mask Maps. Each token is first given a 1-bit Mask flag. Prefix-sum adders then compute, for every row, the number of unmasked tokens (Row Conf.). A block's overall confidence (Block Conf.) is the sum of the Row Conf. within that block. After the Bitonic Sort unit orders the rows inside each block according to their Row Conf., this ranking information is stored in the HTs. During the Formal Sorting phase, we always prioritize the most confident row from the most confident block, as shown in Fig. 13

right. The MAX unit is used to ensure that we only select the most confident block by comparing their Current Block Confidences. Then, the Arbiter asserts the “Hit” signal to the most confident block (e.g., Block #0) and the most confident row in that block indicated by the Max Ptr (e.g., Row 9) is selected. At last, the selected Row & Block IDs are used to generate the address of the selected row. In the next cycle, because the selected row has been consumed, that row’s Row Conf. is subtracted from the corresponding Block Conf. (e.g., “-8”), and the Max Ptr is incremented (an example can be seen in Fig. 13 in the middle bottom). With the inherent advantage of the RS algorithm and the guidance of LAS, RSC yields 4.0x and 5.1x speedups compared to the Mapping unit in [49] and the Topk-engine in [75], respectively. Compared to RSC w/o LAS, RSC w/LAS cuts latency by 23.2% at the cost of a 12.8% area increase.

V. EVALUATION

A. Evaluation Methodology

Algorithm Setup. Models: For the stage 1) Tokenization, we source it from VQGAN [21]. For the stage 2) Generation, we employ a generative Transformer following DeiT [72] (the generalization study on model architectures or datasets can be seen in Sec. V-G). Training Settings: The generative Transformer is trained over 500 epochs with a batch size of 256 on ImageNet, leveraging four V100 GPUs. The loss function we use is cross-entropy loss and the optimizer employed is AdamW with a learning rate of $1e-4$, a weight decay of $1e-5$ and the momentum is (0.9,0.96). The training cost is 816 GPU hours. For hyperparameter optimization, we employ a hybrid strategy: PD-aware training is used for Draft-Free Parallel Decoding (PD) [54], while an efficient grid search is applied to Token Aggregation (TA) and Dynamic Bypass (DB) (see Sec. V-F for details). This approach is motivated by the fact that PD involves multiple hyperparameters, such as sampling temperature, masking ratio, and guidance scale, whose exhaustive search is computationally prohibitive. Therefore, these parameters are adaptively determined during training [54]. In contrast, the hyperparameter spaces for TA and DB are relatively small and enumerable, particularly for TA, making grid search a sufficient and cost-effective choice. Metrics: We use the metric of IS [3] and FID [33] to measure the generation quality.

Hardware Setup. We perform the RTL design for the VAR-Turbo accelerator and synthesize it using Synopsys Design Compiler with TSMC 28nm+HPC 1P8M CMOS technology at the corner of TT_25C. Subsequently, we use DCG and ICC2 to complete the chip’s placement and routing, generating the chip’s layout and netlist, from which we can obtain the chip’s area data. At last, after the DRC and LVS check, we utilize VCS and PrimeTime for post-layout simulation to obtain the on-chip power consumption. Regarding the off-chip DRAM specification, considering the limited number of IO pads, we opt for one $2 \times 64\text{bit}$ HBM2 channel @ 2GHz, which provides 32GB/s bandwidth.

Baselines and Evaluation Metrics. Baselines: To benchmark VAR-Turbo and other SOTA accelerators, we opt for a total of four baselines, including two general platforms: Nvidia V100 and Intel Xeon Platinum 8168 CPU @ 2.70GHz, and two ASIC accelerators: ViTCoD [85] and AdapTiV [84]. Metrics: We evaluate all platforms in terms of latency speedups and energy efficiency. For general platforms, torch.cuda.event API on GPU and time.time API on CPU are used to measure the latencies. Besides, when benchmarking with general platforms, we scale up the VAR-Turbo’s hardware resource to match the peak throughput and bandwidth of V100, following [61]. Specifically, we set the peak throughput to 14 TFLOPS and employ 16 HBM2 channels (i.e., 512GB/s DRAM bandwidth) to model the scale-up VAR-Turbo (see the roofline model in Fig. 16). In terms of power consumption, we adopt pynvml [60] and s-tui [1] to acquire the data for GPU and CPU, respectively. For ASIC accelerators, we develop three dedicated cycle-accurate simulators for VAR-Turbo, ViTCoD, and AdapTiV to evaluate latencies, following the method in [75]. To ensure the correctness, we conduct RTL designs for ViTCoD and AdapTiV and prepare ten test cases with latencies ranging from 1 ms to 10 ms at 1 ms intervals. We then compare the latency results tested on the same test cases from our simulators to those obtained from RTL simulations. The matching rates of latency between simulator and RTL are 0.96 for ViTCoD, 0.94 for AdapTiV, and 0.90 for VAR-Turbo, respectively (i.e., the matching rates are all above 90%). For the on-chip power data, we directly obtain it from the reports of ViTCoD and AdapTiV for a fair comparison. For VAR-Turbo, as previously mentioned, we acquire the power data from the post-layout simulation. For off-chip DRAM energy consumption, we simulate the number of row activation, read/write with Ramulator [41] to calculate the overall energy. It is notable that we adopt three DDR4 to get the two ASIC baselines’ claimed 76.8GB/s DRAM bandwidth.

B. Algorithm Evaluation

Tab. I summarizes the evaluation results for VAR-Turbo against other generative models. Given that the vast majority of other generative models are only designed for 256×256 and/or 512×512 image resolutions, we restrict our analysis to these two cases in this section to ensure a fair comparison. In Tab. I, borrowing the idea of *Logic Effort* in VLSI design [70], we introduce a new metric called *Generation Effort* (*Gen. Effort*), which is defined as “TFLOPs/IS”, to quantify the computational effort per IS. The term **VAR-Turbo-Peak** denotes the highest achievable generation quality for VAR-Turbo. However, this peak point is associated with higher latency. To strike a balance between actual latency and generation quality, we introduce an alternative default option for the accelerator side of VAR-Turbo, i.e., **VAR-Turbo-Balance** (further justifications are provided in Sec. V-F). Overall, VAR-Turbo demonstrates superior generation quality and less Gen. Effort compared to other generative models. Notably, when compared to MaskGIT, which is a pioneering work applying PD on VAR models, VAR-Turbo-Peak exhibits significantly

TABLE I: Comparison with Other Generative Models (TFLOPs:The Number of Computation).

Model	Type	#Para	#Step@256	TFLOPs@256 ↓	FID@256 ↓	IS@256 ↑	Gen. Effort@256 ↓	#Step@512	TFLOPs@512 ↓	FID@512 ↓	IS@512 ↑	Gen. Effort@512 ↓
ADM [15]	Diffusion	554M	16	21.4	5.28	214.8	0.10	16	37.1	8.8	157.2	0.236
LDM [64]	Diffusion	400M	16	3.7	3.68	202.7	0.018	N/A	N/A	N/A	N/A	N/A
U-ViT [2]	Diffusion	500M	16	4.6	2.77	259.5	0.018	16	5.5	4.04	252.6	0.022
DiT-XL [58]	Diffusion	675M	16	4.1	3.13	256.1	0.016	16	18.0	3.84	211.7	0.085
Mask-Diff [25]	Diffusion	676M	8	2.2	4.0	N/A	N/A	N/A	N/A	N/A	N/A	N/A
VQ-GAN [21]	VAR	227M	256	18.7	18.65	80.4	0.23	1024	N/A	26.52	66.8	N/A
MaskGIT [9]	VAR	227M	8	0.6	6.18	182.1	0.044	12	3.3	7.32	156.0	0.021
PAR-XL [78]	VAR	775M	64	21.1	2.64	259.2	0.08	N/A	N/A	N/A	N/A	N/A
VAR-Turbo-Peak	VAR	457M	20	2.8	2.65	272.4	0.01	64	11.5	3.13	263.4	0.04
VAR-Turbo-Balance	VAR	457M	8	1.1	2.67	268.6	0.004	32	5.7	3.15	259.6	0.021

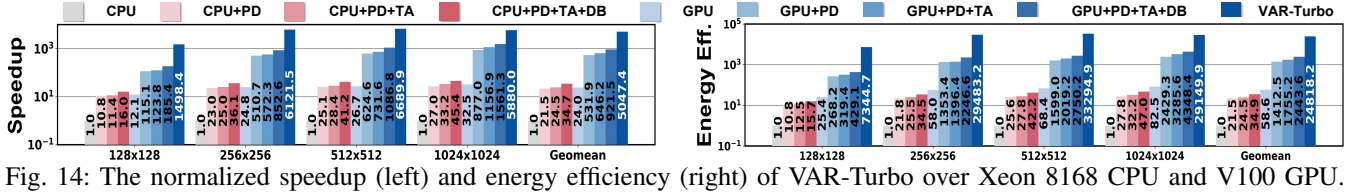


Fig. 14: The normalized speedup (left) and energy efficiency (right) of VAR-Turbo over Xeon 8168 CPU and V100 GPU.

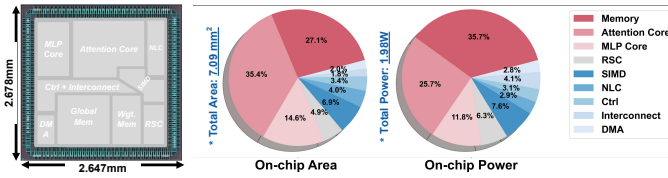


Fig. 15: A layout image of VAR-Turbo and its area/power consumption breakdown results.

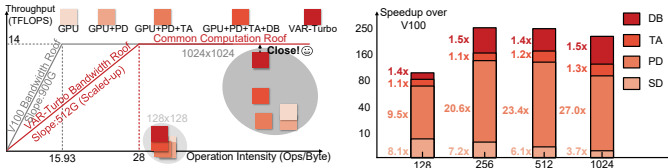


Fig. 16: The Roofline Model (left) and Speedup breakdown (right). SD: Specialized Datapath.

higher quality ($90.3 \uparrow @IS$) and much lower Gen. Effort. The former advantage stems from our PD-aware training strategy. MaskGIT adopts a heuristic policy to fix the hyperparameters for PD, inevitably degrading fidelity. In contrast, we optimize PD’s hyperparameters through a gradient-based approach, i.e., *PD-Aware Training* [55] [54], which brings much higher visual quality by determining PD’s hyperparameters adaptively throughout training. The latter advantage is originated from our proposed TA and DB, which greatly reduce the compute budget. Crucially, our PD is enhanced by the Locality-aware-Scheduling (see Sec. IV-D and Fig. 13), which can maximize the achievable hardware speedup brought by PD. In stark contrast, MaskGIT’s PD is completely hardware-unaware, thereby forgoing any opportunity to optimize hardware efficiency. Then against the SOTA VAR model, PAR, VAR-Turbo delivers comparable quality yet requires $7.5\times$ fewer TFLOPs and $8\times$ fewer Gen. Effort. This gain arises primarily because PAR decodes a fixed number of tokens per iteration, e.g., 4, whereas VAR-Turbo can decode up to 64 tokens per iteration. In addition, TA and DB substantially reduce the compute budget of VAR-Turbo, markedly enhancing its Gen. Effort. In terms of the comparison with the SOTA Diffusion model, U-ViT, VAR-Turbo-Peak shows higher generation quality ($12.9 \uparrow @IS$)

and requires $1.6\times$ fewer TFLOPs while at the cost of similar #Para. At last, VAR-Turbo-Balance can deliver much lower TFLOPs and Gen. Effort ($2.5\times \downarrow$) than VAR-Turbo-Peak with only $\sim 1\%$ quality drop. If VAR-Turbo is set to “VAR-Turbo-Peak”, we can still obtain more than $100\times$ speedup compared to a GPU (see Sec. V-F). Critically, VAR-Turbo’s hardware is designed to accept tunable hyperparameters to prioritize latency or quality.

C. Overall Hardware Characteristics

Fig. 15 shows VAR-Turbo’s layout floorplan and its area/power breakdown. Running at 1GHz, it occupies $7.09mm^2$ silicon area and consumes 1.98W. Attention Core and Memory together dominate the hardware overhead, taking 35.4% and 27.1% of the area, and 25.7% and 35.7% of the power, respectively. In contrast, PD and DB necessitated Radix Sort Core (RSC) uses only 4.9% of the area and 6.3% of the power, evidencing the negligible hardware overhead of PD and DB. Further, if we remove RSC from VAR-Turbo and employ CPU (Xeon 8168) to conduct TopK with the remaining operators handled by VAR-Turbo w/o RSC, the overall speedup over GPU will be degraded from 210x to 153x due to the serial execution nature of CPU and excessive off-chip DRAM access, indicating the necessity and efficiency of dedicated on-chip RSC performing TopK. Finally, based on the current evaluation and the technology scaling data from TSMC ($N28 \Rightarrow N3$, the power is reduced by $3.4\times$), embedding VAR-Turbo into the latest Apple A18 Pro SoC (9W @ TSMC N3E technology node) appears to require an estimated 6.5% energy overhead to enable efficient inference of VAR models.

D. Comparisons with General Platforms

Fig. 14 shows the speedup and energy efficiency comparison of VAR-Turbo with two general platform base-lines across four image resolutions. VAR-Turbo attains $5047.4\times$, $234.7\times$, $206.0\times$, $145.4\times$, $210.3\times$, $9.5\times$, $7.8\times$ and $5.5\times$ speedup, $24818.2\times$, $1154.3\times$, $1013.0\times$, $711.1\times$, $423.5\times$, $17.6\times$, $14.1\times$ and $10.1\times$ energy efficiency on average over CPU, CPU+PD, CPU+PD+TA, CPU+PD+TA+DB, GPU, GPU+PD, GPU+PD+TA and GPU+PD+TA+DB, respectively. These results not only witness the superiority of VAR-Turbo over CPU

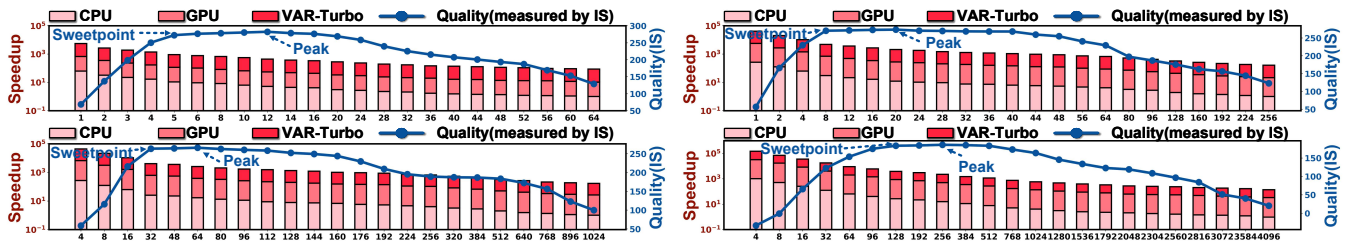


Fig. 17: Design space exploration on generation quality and performance @ PD. Top Left: 128×128 , Top Right: 256×256 , Bottom Left: 512×512 and Bottom Right: 1024×1024 .

TABLE II: Comparisons with Two ASIC Accelerators.

	ViTCoD	AdapTiV	VAR-Turbo
Image Redun.	✗	✓	✓
Model Redun.	✓	✗	✓
Accelerate	Attention	Attention+FFN	Whole Model
Token Opt.	Pruning	Merging	TA+DB
Decoding Type	Serial	Serial	Parallel
Quality Drop	2.9% @ ViT	1.3% @ ViT	1% @ VAR
Technology	28nm	28nm	28nm+1P8M
Circuit State	Layout	Synthesis	Layout
DRAM Spec.	DDR4(76.8GB/s)	DDR4(76.8GB/s)	HBM2(32GB/s)
DRAM Power(W)	8.3	8.3	2.0
Data Format	N/A	FX16	BF16
Frequency(MHz)	500	1000	1000
Area(mm ²)	3	2.49	7.09
Perf.(TOPS)	6.7	10.9	41.2
Power Eff.(TOPS/W)	3.4	2.6	20.3

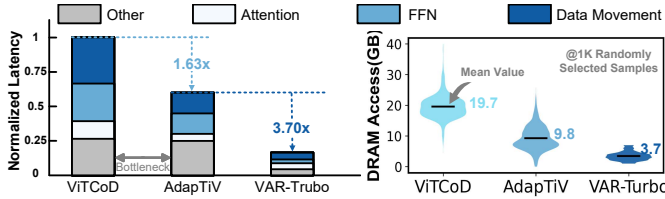


Fig. 18: The latency breakdown of ViTCoD, AdapTiV and VAR-Turbo (left). The statistics of DRAM access (right).

and GPU, but also reflect the cross-platform (CPU,GPU,ASIC) effectiveness of PD, TA and DB.

Fig. 16 summarizes the performance analysis of VAR-Turbo. At first, we use the roofline model to better elucidate the effect of different optimization schemes. Obviously, VAR models are compute-bound tasks for both GPU and VAR-Turbo. Then, since PD introduces extra bandwidth-constraint operators like Top-k and Softmax, the throughput is reduced in the “+PD” case. However, PD scheme greatly decreases the amount of iterations; thus, the overall latency is still much smaller than the vanilla VAR models. TA & DB reduce a certain amount of operations with minor extra parameters, which makes “+TA” and “+DB” cases showcase higher throughput and slightly lower operation intensity. Additionally, in the low image resolution case, the length of visual tokens is smaller than the embedding size, e.g., 128 v.s. 1024 @ 128×128 , which means that the TA scheme is less effective in the low image resolution case (see the light gray elliptical area in Fig. 16 left). At last, VAR-Turbo obtains a much closer throughput gap towards the computation roof over GPU+PD+TA+DB due to its highly specialized datapath (3.7x-8.1x speedup by Specialized Datapath (SD), see Fig.16 right).

E. Comparisons with ASIC Accelerators

Overall Comparisons. To further understand the significance of PD, TA and DB, and validate the necessity of VAR-Turbo, we compare it with two SOTA ViT accelerators, ViTCoD and AdapTiV. This is because there is no prior art targeting VAR models and ViT accelerators are the most relevant baselines that we can choose. Besides, to ensure a fair comparison, we restrict the evaluation of runtime overhead and accuracy loss of the two ASICs to the identical Vision Transformer backbone utilized in the VAR model benchmarking VAR-Turbo. The comparison results are summarized in Tab. II. Besides, in Tab. II, we scale up the hardware resources of ViTCoD and AdapTiV to match those of VAR-Turbo, thereby ensuring a fair comparison in terms of throughput and power efficiency. First, both ASIC baselines only leverage one-sided redundancy. Specifically, ViTCoD adopts static pruning and AdapTiV opts for adaptive token merging. Both of the strategies consistently exhibit a larger drop in the generation quality than VAR-Turbo, with ViTCoD being particularly affected—its static pruning proves not well-suited to the image generation task. Then, VAR-Turbo shows 6.1x, 3.8x higher throughput, 6.0x, 7.8x higher power efficiency over ViTCoD and AdapTiV, respectively. Please note that in Tab. II, to ensure a fair comparison (i.e., under the constraint of similar quality drop), we use the configuration set of “VAR-Turbo-Peak” (see the second paragraph in Sec. V-F) to compare with other two ASIC baselines.

Breakdown Analysis. To elucidate the sources of VAR-Turbo’s efficiency over other two ASIC baselines, we conduct a breakdown analysis which is summarized in Fig. 18. Overall, as shown in Fig. 18 left, VAR-Turbo shows substantially lower end-to-end latency than ViTCoD and AdapTiV. This superiority stems from two factors: 1) VAR-Turbo capitalizes on both image and model redundancy, whereas ViTCoD and AdapTiV exploit only one side of redundancy. Specifically, in terms of image redundancy, PD reduces the number of iterations by 80%, while prior ViT accelerators have to decode tokens serially. The reason is that 1) ViTCoD sacrifices 2.9% accuracy (Tab. II), exhausting any margin left for extra optimizations such as PD. 2) For AdapTiV, since the initial input for VAR models is an all-masked canvas in the inference phase, token merging in AdapTiV will merge numerous masked tokens into one. Thus, if we apply PD for AdapTiV, it will generate the same/similar pixel value for the previously merged masked tokens, greatly degrading the visual quality. Consequently,

neither baseline is compatible with PD. Then, regarding model redundancy, TA & DB reduce the computational load for both attentions and MLPs. In contrast, ViTCoD only optimizes attentions. 2) VAR-Turbo presents a more advanced and customized system-on-chip (SOC) design (e.g., layer fusion, “producer-consumer” SOC schedule, RSC and NLC, see Sec. IV-B), specifically engineered to provide seamless support for the whole VAR model. However, ViTCoD and AdapTiV primarily target optimizing attentions and/or FFNs. Consequently, VAR-Turbo achieves much less DRAM access than other two accelerators (see Fig. 18 right). Besides, for those bandwidth-limited operators (Legend “Other” in Fig. 18), e.g., TopK and non-linear functions, which are also witnessed to be critical in terms of the end-to-end latency [26], they receive almost no specialized optimization in ViTCoD and AdapTiV, thereby constraining their attainable performance considerably.

F. Design Space Exploration

In this section, we search and select the optimal architectural settings for VAR-Turbo via design space exploration (DSE).

PD: Sampling Steps. First and foremost, counter-intuitively, as shown in Fig. 17, under PD scheme, an increased number of iterations may lead to deteriorated generation quality! This finding corroborates the claim in [31] that the relationship between pixels is bidirectional. Excessive iterations tend to weaken this relationship, thereby negatively impacting the overall quality. When prioritizing the quality metric, the optimal number of iterations is 12, 20, 64 and 256 for the image resolutions ranging from 128×128 to 1024×1024 , respectively (Peaks in Fig. 17 or VAR-Turbo-Peak in Sec. V-B). Then, apparently, an increased number of iterations correlates with longer latency. When considering the factor of latency, the Sweetpoints are identified as 5, 8, 32, 128 for the image resolutions spanning from 128×128 to 1024×1024 , respectively; since an acceptable level of quality is achieved (the quality drop $\sim 1\%$) with the minimal latency at these four Sweetpoints (VAR-Turbo-Balance in Sec. V-B, $\sim 200\times$ speedup compared to a GPU when combining with our proposed accelerator, see Fig. 14).

TA: Local Window. The results are summarized in Fig. 19. As shown in Fig. 19, when $LW.size \geq 8$, the generation quality drops drastically for all the image resolutions. Then, we find that when the image resolution goes higher, the tolerance of large $LW.size$ is also growing larger. Therefore, for low-resolution images, such as those with resolutions of 128×128 and 256×256 , we predominantly set the $LW.size$ to 2. In contrast, for high-resolution images, such as those with resolutions of 512×512 and 1024×1024 , we allocate 50% of the $LW.size$ to 2 and the remaining 50% to 4. The generation quality drops for all four image resolutions are below 0.5%.

DB: Skip Rate. The schedule function is the key factor in DB design, which controls the token skip rate layer-wise. In Ref. [54] and Ref. [9], the authors find that concave functions (e.g., Cosine) generally get better quality when scheduling the

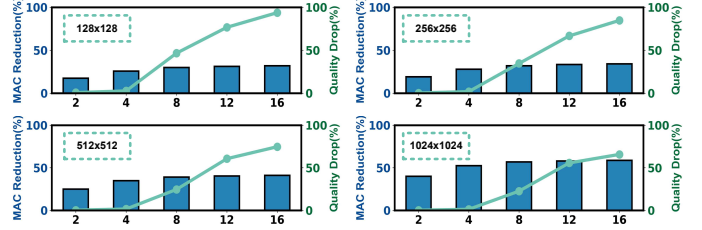


Fig. 19: Design Space Exploration on $LW.size$ @ TA.

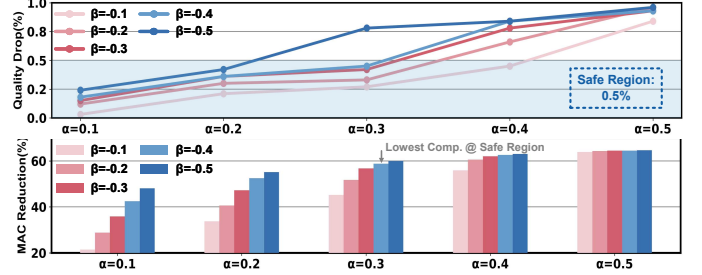


Fig. 20: Design Space Exploration on $\alpha \& \beta$ @ DB.

skip rate or mask rate. Therefore, we set the default schedule function as:

$$S(l) = \min(\alpha + \beta \cos(\frac{\pi(l+16)}{32}), Thre.), l = 0 \Rightarrow 15. \quad (2)$$

In Eq. 2, we empirically set the threshold ($Thre.$) to 0.55, which means that there are 55% tokens can be skipped at most. Then, we conduct a design space exploration to determine the coefficients, i.e., $\alpha \& \beta$. As shown in Fig. 20, when $\alpha = 0.3$ and $\beta = -0.4$, VAR-Turbo achieves a minimal generation quality drop ($< 0.5\%$) while simultaneously maximizing the MAC reduction rate. Thus, we set $\alpha = 0.3$ and $\beta = -0.4$ for the schedule function $S(l)$ in DB.

Ablation Study. Tab. III ablates each optimization’s impact on visual quality and end-to-end (E2E) hardware speedup. Firstly, no single technique or their combination introduces perceptible quality loss, certifying lossless acceleration and their orthogonality. Then, in terms of the E2E hardware speedup, PD dominates the speedup by cutting over 80% iterations. DB and TA furnish sizable incremental speedups: DB bypasses both attention and FFN layers, while TA compresses attention itself. Besides, removing any single component drops speedup over 20%, underscoring their combination’s necessity.

TABLE III: Ablation Study on PD/TA/DB @ 256×256 .

(PD TA DB)	FID ↓	ΔFID	E2E Speedup ↑	Δ Speedup
(X X X)	2.637	-	1	-
(X X ✓)	2.645	+8e-3	1.65x	+0.65x
(X ✓ X)	2.642	+5e-3	1.44x	+0.44x
(✓ X X)	2.662	+2.5e-2	20.4x	+19.4x
(X ✓ ✓)	2.663	+2.6e-2	1.98x	+0.98x
(✓ ✓ X)	2.673	+3.6e-2	30.9x	+29.9x
(✓ ✓ ✓)	2.670	+3.3e-2	28.6x	+27.6x
(✓ ✓ ✓)	2.675	+3.8e-2	35.5x	+34.5x

G. Generalization Study

To rigorously validate the robust generalization of our contributions, we instantiate VAR models with four widely-used

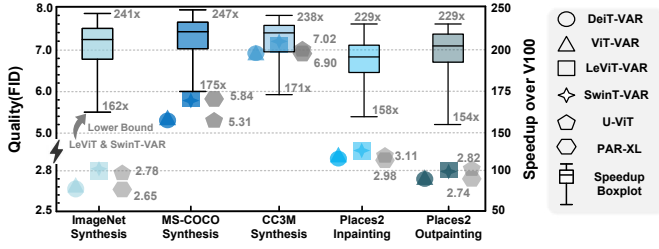


Fig. 21: Generalization analysis of the proposed optimizations. The lower bound of the speedup boxplot is determined by LeViT- and SwinT-VAR. Speedup benchmarks: Comparison between the SW/HW co-design framework and V100.

Vision Transformers and evaluate them on four prevalent datasets: ImageNet, MS-COCO [48], CC3M [68] and Places2 [92] (see Fig. 21). Image synthesis tasks are conducted on the first three datasets; inpainting/outpainting tasks are carried out on Places2. Besides, U-ViT and PAR-XL serve as competitive diffusion and VAR baselines, respectively. Overall, across all backbones, datasets and tasks, our optimized VAR models achieve significant speedups over V100 while maintaining visual quality comparable to the diffusion and VAR baselines. Notably, LeViT-VAR and SwinT-VAR exhibit relatively lower generation quality and speedup compared to ViT-VAR and DeiT-VAR. This discrepancy can be attributed to the presence of extra Conv2D modules within LeViT and SwinT. Conv2D modules will downsample the input tokens with a slight information loss, hurting the quality. Moreover, these two models also need additional 1-2 extra steps to maintain a comparable generation quality, hurting the speedup. Nevertheless, even for LeViT-VAR and SwinT-VAR, they still achieve substantial speedups over V100 while delivering competitive generation quality compared to U-ViT and PAR-XL. Consequently, our innovations exhibit robust generalizability and potentially can serve as a model-agnostic plug-in for future works.

VI. RELATED WORKS AND DISCUSSION

Accelerators for Vision Generative Models. Various hardware accelerators have been proposed since the advent of vision generative models. However, these works predominantly focus on accelerating GANs [13] [82], Diffusion models [42], [81], [93] or just the backbone, Vision Transformers [17], [84], [85]. Their main contributions primarily fall under four categories, 1) Differential Processing [42], 2) Sparsification or Quantization [17], [81], [85], 3) Linear Attention [14], and 4) Redundancy Utilization [84]. However, VAR-Turbo, this work distinguishes itself by pioneering a SW/HW co-design framework that aims to accelerate the entire VAR models, leveraging both image and model redundancy.

Redundancy of Images. A variety of works have made their contributions based on the redundancy of images [6], [9], [10], [17], [23], [31], [62], [84], [88]. For example, AdapTiV picks one representative token from several similar tokens to decrease the number of tokens to be processed, which is inspired by the abundant image redundancy [84]. Alternatively,

PSViT [10] adopts token pooling by leveraging the model redundancy of ViTs. In this study, to the best of our knowledge, VAR-Turbo presents an early attempt to theoretically identify both image redundancy and model redundancy, and correspondingly, enhances the VAR acceleration framework by PD, TA and DB.

VII. CONCLUSION

In this work, we present VAR-Turbo, the first SW/HW co-design framework for visual autoregressive models. Exploiting image- and model-level redundancy, we propose three algorithmic optimizations: Draft-Free Parallel Decoding, Token Aggregation and Dynamic Bypass. To maximize the achievable speedup brought by SW innovations, in the hardware level, VAR-Turbo incorporates 1) A Unified Attention Core to accommodate distinct attentions introduced by TA and 2) Radix Sort Core to manage the large-scale sorting operation introduced by PD and DB. Under synergistic effort between SW and HW, VAR-Turbo delivers over 200 $\times$ speedup for VAR models while incurring negligible quality degradation, suggesting a promising direction for future research in Generative-AI accelerators.

VIII. ACKNOWLEDGEMENT

This research was supported in part by NSFC Grant 62422407, RGC Grant 26204424, and partially conducted by ACCESS – AI Chip Center for Emerging Smart Systems, supported by the InnoHK initiative of the Innovation and Technology Commission of the Hong Kong Special Administrative Region Government. We are grateful to Dr. Zhiheng Yue and the anonymous reviewers for their insightful comments, which substantially improved the quality of this paper.

REFERENCES

- [1] Amanusk, “The stress terminal ui: s-tui,” [Online], <https://amanusk.github.io/s-tui/>.
- [2] F. Bao, S. Nie, K. Xue, Y. Cao, C. Li, H. Su, and J. Zhu, “All are worth words: A vit backbone for diffusion models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2023, pp. 22 669–22 679.
- [3] S. Barratt and R. Sharma, “A note on the inception score,” *arXiv preprint arXiv:1801.01973*, 2018.
- [4] I. Beltagy, M. E. Peters, and A. Cohan, “Longformer: The long-document transformer,” *arXiv preprint arXiv:2004.05150*, 2020.
- [5] Black Forest Labs, “black-forest-labs/flux: Official inference repo for flux.1 models,” <https://github.com/black-forest-labs/flux>, accessed: 2025-07-25.
- [6] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, C. Feichtenhofer, and J. Hoffman, “Token merging: Your vit but faster,” *arXiv preprint arXiv:2210.09461*, 2022.
- [7] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, “Medusa: Simple llm inference acceleration framework with multiple decoding heads,” *arXiv preprint arXiv:2401.10774*, 2024.
- [8] Y. Cao, S. Li, Y. Liu, Z. Yan, Y. Dai, P. S. Yu, and L. Sun, “A comprehensive survey of ai-generated content (aigc): A history of generative ai from gan to chatgpt,” *arXiv preprint arXiv:2303.04226*, 2023.
- [9] H. Chang, H. Zhang, L. Jiang, C. Liu, and W. T. Freeman, “Maskgit: Masked generative image transformer,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 11 315–11 325.
- [10] B. Chen, P. Li, B. Li, C. Li, L. Bai, C. Lin, M. Sun, J. Yan, and W. Ouyang, “Psvit: Better vision transformer via token pooling and attention sharing,” *arXiv preprint arXiv:2108.03428*, 2021.

- [11] M. Chen, A. Radford, R. Child, J. Wu, H. Jun, D. Luan, and I. Sutskever, "Generative pretraining from pixels," in *International conference on machine learning*. PMLR, 2020, pp. 1691–1703.
- [12] Y. Chen, X. Pan, Y. Li, B. Ding, and J. Zhou, "Ee-llm: Large-scale training and inference of early-exit large language models with 3d parallelism," *arXiv preprint arXiv:2312.04916*, 2023.
- [13] Y. Chen, A. Louri, F. Lombardi, and S. Liu, "Chiplet-gan: Chiplet-based accelerator design for scalable generative adversarial network inference [feature]," *IEEE Circuits and Systems Magazine*, vol. 24, no. 3, pp. 19–33, 2024.
- [14] J. Dass, S. Wu, H. Shi, C. Li, Z. Ye, Z. Wang, and Y. Lin, "Vitality: Unifying low-rank and sparse approximation for vision transformer acceleration with a linear taylor attention," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 415–428.
- [15] P. Dhariwal and A. Nichol, "Diffusion models beat gans on image synthesis," *Advances in neural information processing systems*, vol. 34, pp. 8780–8794, 2021.
- [16] G. Dimitrakopoulos and D. Nikolos, "High-speed parallel-prefix vlsi ling adders," *IEEE Transactions on Computers*, vol. 54, no. 2, pp. 225–231, 2005.
- [17] P. Dong, M. Sun, A. Lu, Y. Xie, K. Liu, Z. Kong, X. Meng, Z. Li, X. Lin, Z. Fang *et al.*, "Heatvit: Hardware-efficient adaptive token pruning for vision transformers," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 442–455.
- [18] X. Dong, J. Bao, T. Zhang, D. Chen, W. Zhang, L. Yuan, D. Chen, F. Wen, and N. Yu, "Bootstrapped masked autoencoders for vision bert pretraining," in *European Conference on Computer Vision*. Springer, 2022, pp. 247–264.
- [19] Z. Du, Y. Qian, X. Liu, M. Ding, J. Qiu, Z. Yang, and J. Tang, "Glm: General language model pretraining with autoregressive blank infilling," *arXiv preprint arXiv:2103.10360*, 2021.
- [20] J. Duan, S. Yu, H. L. Tan, H. Zhu, and C. Tan, "A survey of embodied ai: From simulators to research tasks," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 6, no. 2, pp. 230–244, 2022.
- [21] P. Esser, R. Rombach, and B. Ommer, "Taming transformers for high-resolution image synthesis," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 12 873–12 883.
- [22] F. Farshchi, Q. Huang, and H. Yun, "Integrating nvidia deep learning accelerator (nvla) with risc-v soc on firesim," in *2019 2nd Workshop on Energy Efficient Machine Learning and Cognitive Computing for Embedded Applications (EMC2)*. IEEE, 2019, pp. 21–25.
- [23] Z. Feng and S. Zhang, "Efficient vision transformer via token merger," *IEEE Transactions on Image Processing*, 2023.
- [24] Y. Fu, P. Bailis, I. Stoica, and H. Zhang, "Break the sequential dependency of llm inference using lookahead decoding," *arXiv preprint arXiv:2402.02057*, 2024.
- [25] S. Gao, P. Zhou, M.-M. Cheng, and S. Yan, "Masked diffusion transformer is a strong image synthesizer," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2023, pp. 23 164–23 173.
- [26] S. Ghodrati, S. Kinzer, H. Xu, R. Mahapatra, Y. Kim, B. H. Ahn, D. K. Wang, L. Karthikeyan, A. Yazdanbakhsh, J. Park, N. S. Kim, and H. Esmaeilzadeh, "Tandem processor: Grappling with emerging operators in neural networks," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1165–1182. [Online]. Available: <https://doi.org/10.1145/3620665.3640365>
- [27] D. Ghosh, H. Hajishirzi, and L. Schmidt, "Geneval: An object-focused framework for evaluating text-to-image alignment," *Advances in Neural Information Processing Systems*, vol. 36, pp. 52 132–52 152, 2023.
- [28] T. GLM, A. Zeng, B. Xu, B. Wang, C. Zhang, D. Yin, D. Zhang, D. Rojas, G. Feng, H. Zhao *et al.*, "Chatglm: A family of large language models from glm-130b to glm-4 all tools," *arXiv preprint arXiv:2406.12793*, 2024.
- [29] A. Gupta, S. Savarese, S. Ganguli, and L. Fei-Fei, "Embodied intelligence via learning and evolution," *Nature communications*, vol. 12, no. 1, p. 5721, 2021.
- [30] S. Hassani, "Dirac delta function," in *Mathematical Methods: For Students of Physics and Related Fields*. Springer, 2009, pp. 289–319.
- [31] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 16 000–16 009.
- [32] T. Henighan, J. Kaplan, M. Katz, M. Chen, C. Hesse, J. Jackson, H. Jun, T. B. Brown, P. Dhariwal, S. Gray *et al.*, "Scaling laws for autoregressive generative modeling," *arXiv preprint arXiv:2010.14701*, 2020.
- [33] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "Gans trained by a two time-scale update rule converge to a local nash equilibrium," *Advances in neural information processing systems*, vol. 30, 2017.
- [34] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.
- [35] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford *et al.*, "Gpt-4o system card," *arXiv preprint arXiv:2410.21276*, 2024.
- [36] D. Israel, G. V. d. Broeck, and A. Grover, "Accelerating diffusion llms via adaptive parallel decoding," *arXiv preprint arXiv:2506.00413*, 2025.
- [37] J.-W. Jang, S. Lee, D. Kim, H. Park, A. S. Ardestani, Y. Choi, C. Kim, Y. Kim, H. Yu, H. Abdel-Aziz, J.-S. Park, H. Lee, D. Lee, M. W. Kim, H. Jung, H. Nam, D. Lim, S. Lee, J.-H. Song, S. Kwon, J. Hassoun, S. Lim, and C. Choi, "Sparsity-aware and re-configurable npu architecture for samsung flagship mobile soc," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 15–28.
- [38] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th annual international symposium on computer architecture*, 2017, pp. 1–12.
- [39] S.-C. Kao, S. Subramanian, G. Agrawal, A. Yazdanbakhsh, and T. Krishna, "Flat: An optimized dataflow for mitigating attention bottlenecks," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 295–310.
- [40] W.-S. Khwa, P.-C. Wu, J.-J. Wu, J.-W. Su, H.-Y. Chen, Z.-E. Ke, T.-C. Chiu, J.-M. Hsu, C.-Y. Cheng, Y.-C. Chen, C.-C. Lo, R.-S. Liu, C.-C. Hsieh, K.-T. Tang, and M.-F. Chang, "34.2 a 16nm 96kb integer/floating-point dual-mode-gain-cell-computing-in-memory macro achieving 73.3-163.3tops/w and 33.2-91.2tflops/w for ai-edge devices," in *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 67, 2024, pp. 568–570.
- [41] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible dram simulator," *IEEE Computer architecture letters*, vol. 15, no. 1, pp. 45–49, 2015.
- [42] W. Kong, Y. Hao, Q. Guo, Y. Zhao, X. Song, X. Li, M. Zou, Z. Du, R. Zhang, C. Liu *et al.*, "Cambricon-d: Full-network differential acceleration for diffusion models," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 903–914.
- [43] W. Kool, H. Van Hoof, and M. Welling, "Stochastic beams and where to find them: The Gumbel-top-k trick for sampling sequences without replacement," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 09–15 Jun 2019, pp. 3499–3508. [Online]. Available: <https://proceedings.mlr.press/v97/kool19a.html>
- [44] A. R. Lahitani, A. E. Permanasari, and N. A. Setiawan, "Cosine similarity to determine similarity measure: Study case in online essay assessment," in *2016 4th International conference on cyber and IT service management*. IEEE, 2016, pp. 1–6.
- [45] C. E. Leiserson, "Fat-trees: Universal networks for hardware-efficient supercomputing," *IEEE transactions on Computers*, vol. 100, no. 10, pp. 892–901, 1985.
- [46] H. Li, Z. Li, Z. Bai, and T. Mitra, "Asadi: Accelerating sparse attention using diagonal-based in-situ computing," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 774–787.
- [47] Y. Liao, J. Meng, and J.-s. Seo, "A 28nm scalable and flexible accelerator for sparse transformer models," in *Proceedings of the 29th ACM/IEEE International Symposium on Low Power Electronics and Design*, 2024, pp. 1–6.
- [48] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft coco: Common objects in

- context,” in *European conference on computer vision*. Springer, 2014, pp. 740–755.
- [49] Y. Lin, Z. Zhang, H. Tang, H. Wang, and S. Han, “Pointacc: Efficient point cloud accelerator,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 449–461.
 - [50] Y.-C. Lo and R.-S. Liu, “Bucket getter: A bucket-based processing engine for low-bit block floating point (bfp) dnns,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 1002–1015.
 - [51] L. Lu, Y. Jin, H. Bi, Z. Luo, P. Li, T. Wang, and Y. Liang, “Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 977–991.
 - [52] E. Malach, “Auto-regressive next-token predictors are universal learners,” *arXiv preprint arXiv:2309.06979*, 2023.
 - [53] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wang, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, “Specinfer: Accelerating large language model serving with tree-based speculative inference and verification,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 932–949. [Online]. Available: <https://doi.org/10.1145/3620666.3651335>
 - [54] Z. Ni, Y. Wang, R. Zhou, J. Guo, J. Hu, Z. Liu, S. Song, Y. Yao, and G. Huang, “Revisiting non-autoregressive transformers for efficient image synthesis,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 7007–7016.
 - [55] Z. Ni, Y. Wang, R. Zhou, R. Lu, J. Guo, J. Hu, Z. Liu, Y. Yao, and G. Huang, “Adanat: Exploring adaptive policy for token-based image generation,” in *European Conference on Computer Vision*. Springer, 2024, pp. 302–319.
 - [56] A. Q. Nichol and P. Dhariwal, “Improved denoising diffusion probabilistic models,” in *International conference on machine learning*. PMLR, 2021, pp. 8162–8171.
 - [57] W. Niu, J. Guan, Y. Wang, G. Agrawal, and B. Ren, “Dnnfusion: accelerating deep neural networks execution with advanced operator fusion,” in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2021, pp. 883–898.
 - [58] W. Peebles and S. Xie, “Scalable diffusion models with transformers,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 4195–4205.
 - [59] D. Podell, Z. English, K. Lacey, A. Blattmann, T. Dockhorn, J. Müller, J. Penna, and R. Rombach, “Sdxl: Improving latent diffusion models for high-resolution image synthesis,” *arXiv preprint arXiv:2307.01952*, 2023.
 - [60] PyPI, “Python utilities for the nvidia management library,” [Online], <https://pypi.org/project/pynvml/>.
 - [61] Y. Qin, Y. Wang, D. Deng, Z. Zhao, X. Yang, L. Liu, S. Wei, Y. Hu, and S. Yin, “Fact: Ffn-attention co-optimized transformer architecture with eager correlation prediction,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–14.
 - [62] Y. Rao, W. Zhao, B. Liu, J. Lu, J. Zhou, and C.-J. Hsieh, “Dynamicvit: Efficient vision transformers with dynamic token sparsification,” *Advances in neural information processing systems*, vol. 34, pp. 13 937–13 949, 2021.
 - [63] Y. Z. Rawash, B. Al-Naami, A. Alfraihat, and H. A. Owida, “Advanced low-pass filters for signal processing: A comparative study on gaussian, mittag-leffler, and savitzky-golay filters,” *Mathematical Modelling of Engineering Problems*, vol. 11, no. 7, 2024.
 - [64] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 10 684–10 695.
 - [65] T. Salimans, A. Karpathy, X. Chen, and D. P. Kingma, “Pixelcnn++: Improving the pixelcnn with discretized logistic mixture likelihood and other modifications,” *arXiv preprint arXiv:1701.05517*, 2017.
 - [66] A. Shanbhag, H. Pirk, and S. Madden, “Efficient top-k query processing on massively parallel hardware,” in *Proceedings of the 2018 International Conference on Management of Data*, ser. SIGMOD ’18. New York, NY, USA: Association for Computing Machinery, 2018, p. 1557–1570. [Online]. Available: <https://doi.org/10.1145/3183713.3183735>
 - [67] C. E. Shannon, “A mathematical theory of communication,” *The Bell system technical journal*, vol. 27, no. 3, pp. 379–423, 1948.
 - [68] P. Sharma, N. Ding, S. Goodman, and R. Soricut, “Conceptual captions: A cleaned, hypernymed, image alt-text dataset for automatic image captioning,” in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2018, pp. 2556–2565.
 - [69] E. Stehle and H.-A. Jacobsen, “A memory bandwidth-efficient hybrid radix sort on gpus,” in *Proceedings of the 2017 ACM International Conference on Management of Data*, 2017, pp. 417–432.
 - [70] I. E. Sutherland and R. F. Sproull, “Logical effort: Designing for speed on the back of an envelope,” *IEEE Advanced Research in VLSI*, vol. 116, p. 270, 1991.
 - [71] E. Swartzlander, “Parallel counters,” *IEEE Transactions on Computers*, vol. C-22, no. 11, pp. 1021–1024, 1973.
 - [72] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, “Training data-efficient image transformers & distillation through attention,” in *International conference on machine learning*. PMLR, 2021, pp. 10 347–10 357.
 - [73] A. Tsirikoglou, G. Eilertsen, and J. Unger, “A survey of image synthesis methods for visual machine learning,” in *Computer graphics forum*, vol. 39, no. 6. Wiley Online Library, 2020, pp. 426–451.
 - [74] A. Van Den Oord, N. Kalchbrenner, and K. Kavukcuoglu, “Pixel recurrent neural networks,” in *International conference on machine learning*. PMLR, 2016, pp. 1747–1756.
 - [75] H. Wang, Z. Zhang, and S. Han, “Spatten: Efficient sparse attention architecture with cascade token and head pruning,” in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 97–110.
 - [76] H.-Y. Wang and T.-S. Chang, “Row-wise accelerator for vision transformer,” in *2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. IEEE, 2022, pp. 399–402.
 - [77] H. Wang, J. Fang, X. Tang, Z. Yue, J. Li, Y. Qin, S. Guan, Q. Yang, Y. Wang, C. Li *et al.*, “Sofa: A compute-memory optimized sparsity accelerator via cross-stage coordinated tiling,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 1247–1263.
 - [78] Y. Wang, S. Ren, Z. Lin, Y. Han, H. Guo, Z. Yang, D. Zou, J. Feng, and X. Liu, “Parallelized autoregressive visual generation,” in *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, June 2025, pp. 12 955–12 965.
 - [79] C. Wu, H. Zhang, S. Xue, Z. Liu, S. Diao, L. Zhu, P. Luo, S. Han, and E. Xie, “Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding,” *arXiv preprint arXiv:2505.22618*, 2025.
 - [80] Z. Yan, J. Ye, W. Li, Z. Huang, S. Yuan, X. He, K. Lin, J. He, C. He, and L. Yuan, “Gpt-imgeval: A comprehensive benchmark for diagnosing gpt4o in image generation,” *arXiv preprint arXiv:2504.02782*, 2025.
 - [81] G. Yang, Y. Xie, Z. J. Xue, S.-E. Chang, Y. Li, P. Dong, J. Lei, W. Xie, Y. Wang, X. Lin *et al.*, “Sda: Low-bit stable diffusion acceleration on edge fpgas,” in *2024 34th International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2024, pp. 264–273.
 - [82] A. Yazdanbakhsh, M. Brzozowski, B. Khaleghi, S. Ghodrati, K. Samadi, N. S. Kim, and H. Esmailzadeh, “Flexigan: An end-to-end solution for fpga acceleration of generative adversarial networks,” in *2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2018, pp. 65–72.
 - [83] S. Yin, C. Fu, S. Zhao, K. Li, X. Sun, T. Xu, and E. Chen, “A survey on multimodal large language models,” *National Science Review*, p. nwae403, 2024.
 - [84] S. Yoo, H. Kim, and J.-Y. Kim, “Adaptiv: Sign-similarity based image-adaptive token merging for vision transformer acceleration,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 64–77.
 - [85] H. You, Z. Sun, H. Shi, Z. Yu, Y. Zhao, Y. Zhang, C. Li, B. Li, and Y. Lin, “Vitcod: Vision transformer acceleration via dedicated algorithm and accelerator co-design,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 273–286.
 - [86] J. Yu, J. Park, S. Park, M. Kim, S. Lee, D. H. Lee, and J. Choi, “Nn-lut: neural approximation of non-linear operations for efficient transformer inference,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 577–582.

- [87] J. Yue, M. Zhan, Z. Wang, Y. He, Y. Li, S. Yu, W. Sun, L. Jie, C. Dou, X. Li *et al.*, "A 5.6-89.9 tops/w heterogeneous computing-in-memory soc with high-utilization producer-consumer architecture and high-frequency read-free cim macro," in *2023 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. IEEE, 2023, pp. 1–2.
- [88] Z. Yue, H. Wang, J. Fang, J. Deng, G. Lu, F. Tu, R. Guo, Y. Li, Y. Qin, Y. Wang *et al.*, "Exploiting similarity opportunities of emerging vision ai models on hybrid bonding architecture," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 396–409.
- [89] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang *et al.*, "Big bird: Transformers for longer sequences," *Advances in neural information processing systems*, vol. 33, pp. 17 283–17 297, 2020.
- [90] F. Zhan, Y. Yu, R. Wu, J. Zhang, S. Lu, L. Liu, A. Kortylewski, C. Theobalt, and E. Xing, "Multimodal image synthesis and editing: A survey and taxonomy," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.
- [91] G. Zhang, N. Attaluri, J. S. Emer, and D. Sanchez, "Gamma: Leveraging gustavson's algorithm to accelerate sparse matrix multiplication," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 687–701.
- [92] B. Zhou, A. Lapedriza, A. Khosla, A. Oliva, and A. Torralba, "Places: A 10 million image database for scene recognition," *IEEE transactions on pattern analysis and machine intelligence*, vol. 40, no. 6, pp. 1452–1464, 2017.
- [93] H. Zhou, Y. Liu, H. Wang, E. Tang, S. Li, Y. Zhang, and K. Wang, "Sdacc: A stable diffusion accelerator on fpga via software-hardware co-design," in *2024 IEEE 32nd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2024, pp. 214–214.